SPARK Academy 2026 - Team Rwanda

Resources Review: Method Papers, GitHub Repositories, Baseline Model & Starter Code

---

1. Recommended Method Papers

The papers below are directly relevant to the technical tasks in the project: object detection on blood-smear images, YOLO architecture variants, spatial analysis of detection errors, and evaluation of false positives in medical imaging.

1.1 YOLOv12, Attention-Centric Real-Time Object Detectors (NeurIPS 2025)

YOLOv12 introduces an Area Attention mechanism and Residual Efficient Layer Aggregation Networks (R-ELAN) to combine the speed of CNN-based YOLO models with the representational power of attention. The nano variant (YOLOv12n) achieves 40.6% mAP on COCO at 1.64 ms on a T4 GPU, making it practical for training on modest hardware.

Paper (arXiv / NeurIPS 2025): https://arxiv.org/abs/2502.12524

GitHub: https://github.com/sunsmarterjie/yolov12

Ultralytics docs: https://docs.ultralytics.com/models/yolo12/

1.2 Ultralytics YOLOv8, Official Documentation and Technical Report (2023)

Documentation and code:

Docs: https://docs.ultralytics.com/models/yolov8/

GitHub: https://github.com/ultralytics/ultralytics

1.3 Automated Multi-Model Framework for Malaria Detection Using Deep Learning (Scientific Reports, 2025)

This is classification.

Paper (Nature Scientific Reports): https://www.nature.com/articles/s41598-025-04784-w

1.4 YOLO-PAM: Parasite-Attention-Based Model for Efficient Malaria Detection

Paper (MDPI): YOLO-PAM PAPER

GitHub: https://github.com/Snarci/YOLO-SPAM

1.5 YOLO-SPAM: Small-Parasite-Attention-Based Model for Efficient Malaria Detection

Paper: YOLO SPAM PAPER

GitHub: https://github.com/Snarci/YOLO-SPAM

1.6 Supporting Malaria Diagnosis Using Deep Learning and Data Augmentation (MDPI Diagnostics, 2024)

This paper specifically addresses malaria parasite detection in thick smears with deep learning and examines how data augmentation affects model generalisation. The augmentation strategies discussed here can be applied when preparing training configuration for both models.

Paper (MDPI Diagnostics): https://www.mdpi.com/2075-4418/14/7/690

1.7 Confidence Score: The Forgotten Dimension of Object Detection Performance Evaluation (MDPI Sensors, 2021)

This paper is an essential background for Objective 2. It examines how confidence score thresholds interact with false positive rates in object detectors, and why standard mAP metrics can conceal high false positive rates at operating thresholds used clinically. Understanding this paper will help explain why precision, not just mAP, is the correct primary metric for this project, and how to set the confidence threshold when counting FPs in the test-set evaluation.

Paper (MDPI Sensors): https://www.mdpi.com/1424-8220/21/13/4350

1.8 DBSCAN, Density-Based Spatial Clustering of Applications with Noise (Ester et al., 1996)

DBSCAN is the spatial clustering algorithm recommended for Objective 3. Given a set of FP prediction centers and annotated WBC centers, DBSCAN can identify whether FPs cluster densely around WBC locations (supporting the hypothesis) or are spread uniformly across images (contradicting it). It handles clusters of arbitrary shape without requiring the number of clusters to be specified in advance, making it suitable for this exploratory spatial analysis. The Ultralytics glossary entry provides a practical introduction specific to object detection contexts.

Ultralytics DBSCAN glossary:
https://www.ultralytics.com/glossary/dbscan-density-based-spatial-clustering-of-applications-with-noise

Python implementation: https://github.com/sumony2j/DBSCAN_Clustering

2. Recommended GitHub Repositories

Repositories marked with [Direct Use] provide code we can adapt directly. Those marked with [Reference] are useful to read for design decisions.

2.1 Core YOLO Infrastructure

- **ultralytics/ultralytics [Direct Use]** https://github.com/ultralytics/ultralytics

  The main Ultralytics repository contains YOLOv8 through YOLOv12. Install with pip install ultralytics. This provides the training loop, validation, inference, and prediction export functions

you will use throughout this project. The model.predict() function returns Results objects containing all predicted boxes, which are required for FP analysis.

- **sunsmarterjie/yolov12 [Reference]** https://github.com/sunsmarterjie/yolov12

  Original YOLOv12 research repository (NeurIPS 2025). Contains the architecture definition, pre-trained weights, and training scripts for the attention-centric YOLO models. Read the README for architecture notes; use the Ultralytics integration for actual training.

2.2 Malaria Object Detection Repositories

- **samehaisaa/AI-Malaria-Detection-YOLOv8-SVM [Reference]**
  https://github.com/samehaisaa/AI-Malaria-Detection-YOLOv8-SVM

  YOLOv8 + SVM hybrid system for malaria parasite detection in thick smears (2024). Useful to read for the pre-processing pipeline: how annotation files are loaded, how the YOLO format is prepared from raw annotations, and how inference results are post-processed.

- **prtm1908/Blood-Cell-Count-and-Detection-YOLOv8 [Reference]**
  https://github.com/prtm1908/Blood-Cell-Count-and-Detection-YOLOv8

  YOLOv8-based blood cell detection (RBC, WBC, Platelets). While designed for thin smears rather than thick Giemsa smears, this repository demonstrates the multi-class blood cell detection setup in YOLOv8, relevant to Model B (parasite + WBC two-class detector). Read the data.yaml and training script structure.

2.3 False Positive Analysis Utilities

- **ultralytics/ultralytics issue #5367 [Reference]** https://github.com/ultralytics/ultralytics/issues/5367

  Community discussion on how to extract and identify false positive and false negative images from a test dataset using the Ultralytics API. Shows how to use model.val() with save_json=True and how to parse the results to classify individual predictions as TP/FP/FN. This is the starting point for the FP extraction step in Objective 2.

- **sumony2j/DBSCAN_Clustering [Direct Use]** https://github.com/sumony2j/DBSCAN_Clustering

  Clean Python implementation of DBSCAN spatial clustering using sklearn. Can be used directly to cluster FP prediction coordinates relative to WBC center coordinates for Objective 3. The core call is: from sklearn.cluster import DBSCAN; DBSCAN(eps=50, min_samples=2).fit(coords).

3.4 Training Template Repository

- **woodsj1206/Train-Yolov8-Object-Detection-On-Custom-Dataset [Direct Use]**
  https://github.com/woodsj1206/Train-Yolov8-Object-Detection-On-Custom-Dataset

  A clean guide and template for training YOLOv8 on a custom dataset with Google Colab. Covers the data.yaml format, folder structure (images/train, images/val, labels/train, labels/val), and the model.train() call. Use this as a reference when setting up the Lacuna dataset directory structure before training Model A and Model B.

3. Starter Code Resources

The resources below provide runnable code to start training and evaluation immediately. Recommended sequence: start with the Roboflow Colab notebook to train an initial model on the Lacuna dataset, then adapt the Ultralytics prediction extraction pattern for the FP analysis.

3.1 Roboflow, Train YOLO on Custom Dataset [Recommended Starting Point]

Colab Notebook:
https://colab.research.google.com/github/roboflow-ai/notebooks/blob/main/notebooks/train-yolov8-object-detection-on-custom-dataset.ipynb

Blog guide: https://blog.roboflow.com/how-to-train-yolov8-on-a-custom-dataset/

3.2 Lacuna Malaria Detection Challenge, Kaggle Notebooks

The Zindi Lacuna Malaria Detection Challenge closed in November 2024 (1,322 participants). Kaggle mirrors of the dataset and community notebooks are available. These notebooks are useful to understand how the Lacuna annotation format is structured and how other participants approached the same detection problem.

Zindi competition page: https://zindi.africa/competitions/lacuna-malaria-detection-challenge

Kaggle dataset mirror:
https://www.kaggle.com/datasets/rajsahu2004/lacuna-malaria-detection-dataset

Community notebook:
https://www.kaggle.com/code/ronnieleon76/lacuna-malaria-detection-challenge

3.3 Ultralytics, Official Training in Google Colab (Blog + Docs)

The official Ultralytics tutorial for training YOLOv8 in Google Colab. Compared to the Roboflow notebook, this one is closer to the raw Ultralytics API and does not require a Roboflow account. Also read the official Ultralytics predict mode documentation to understand how to extract structured prediction output (boxes, confidence scores, class IDs) for the FP analysis pipeline.

Colab training blog:
https://www.ultralytics.com/blog/training-custom-datasets-with-ultralytics-yolov8-in-google-colab

Predict mode docs: https://docs.ultralytics.com/modes/predict/

3.4 Key Code Pattern: Extracting Predictions for FP Analysis

The following Python code skeleton shows how to run inference on the test set and extract all raw prediction boxes from YOLOv8/YOLOv12 for subsequent TP/FP classification. This pattern is needed for Objectives 2 and 3.

```
from ultralytics import YOLO
import pandas as pd
```

```
model = YOLO('runs/detect/train/weights/best.pt')
results = model.predict(source='data/test/images', save=False,
conf=0.25)

records = []
for r in results:
    img_name = r.path
    for box in r.boxes:
        records.append({
            'image':  img_name,
            'x_ctr':  float(box.xywh[0][0]),
            'y_ctr':  float(box.xywh[0][1]),
            'w':      float(box.xywh[0][2]),
            'h':      float(box.xywh[0][3]),
            'conf':   float(box.conf[0]),
            'cls':    int(box.cls[0]),
        })

df = pd.DataFrame(records)
# Next step: match df rows against ground-truth annotations at IoU
>= 0.5
# to label each row as TP or FP.
```

3.5 Key Code Pattern: Distance-to-Nearest-WBC Analysis (Objective 3)

Once FP predictions and annotated WBC centers are available as NumPy arrays, the spatial distance analysis reduces to the following pattern using scipy and sklearn:

```
import numpy as np
from scipy.spatial import cKDTree
from scipy.stats import mannwhitneyu

# fp_centers: (N, 2) array of (x, y) for each FP prediction in test
set
# tp_centers: (M, 2) array of (x, y) for each TP prediction
# wbc_centers: (K, 2) array of annotated WBC centers (per-image,
looped)

tree = cKDTree(wbc_centers)
fp_dists, _ = tree.query(fp_centers)
tp_dists, _ = tree.query(tp_centers)

stat, p_value = mannwhitneyu(fp_dists, tp_dists, alternative='less')
print(f'FP mean dist to WBC: {fp_dists.mean():.1f} px')
print(f'TP mean dist to WBC: {tp_dists.mean():.1f} px')
print(f'Mann-Whitney p-value: {p_value:.4f}')
```